

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin

NIM. 2010817110011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2024**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN II
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin
NIM : 2010817110011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom., M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

LEMBAR PENGESAHAN	1
DAFTAR ISI	2
DAFTAR GAMBAR	3
DAFTAR TABEL	4
SOAL 1	5
A. Source Code	8
Compose	8
XML	10
B. Output Program	14
C. Pembahasan	15
Compose	15
XML	15
D. Tautan Git	16

DAFTAR GAMBAR

Gambar 1. Contoh Penggunaan Debugger	5
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose	14
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose	15
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose	15
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose	16
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML	17
Gambar 6. Screenshot Break point dan debugger	17
Gambar 7. Screenshot debugger step out	17
Gambar 8. Screenshot debugger onclick	17
Gambar 9. Screenshot debugger onclick stepinto	18

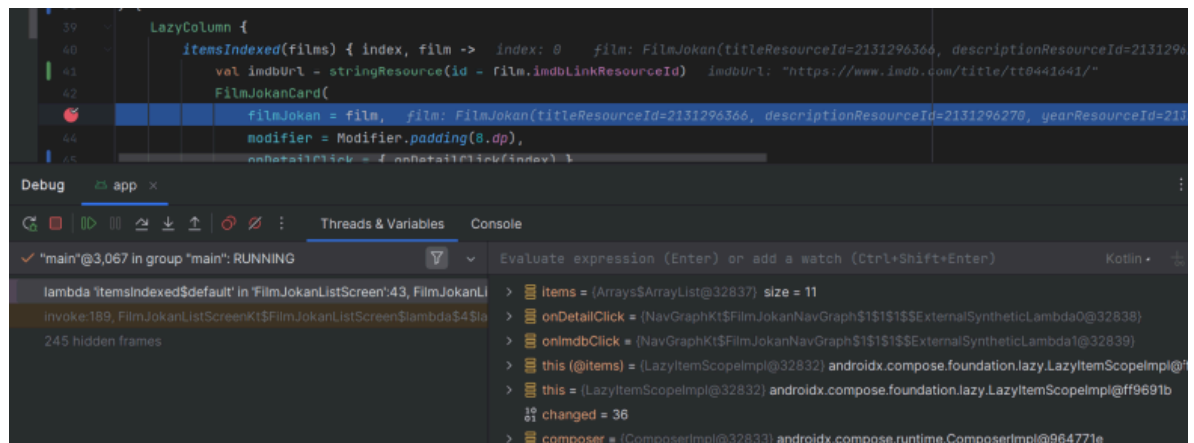
DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1	6
Tabel 2. Source Code Jawaban Soal 1	7
Tabel 3. Source Code Jawaban Soal 1	9
Tabel 4. Source Code Jawaban Soal 1	10
Tabel 5. Source Code Jawaban Soal 1	11
Tabel 6. Source Code Jawaban Soal 1	12
Tabel 7. Source Code Jawaban Soal 1	13

SOAL

Soal Praktikum:

1. Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
 - b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel.
 - c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment.
 - d. Install dan gunakan library Timber untuk logging event berikut:
 - i. Log saat data item masuk ke dalam list
 - ii. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - iii. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out
2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya. Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 1. Contoh Penggunaan Debugger

A. Source Code

Compose MovieListViewModel.kt

```
1 package com.example.movielist.ui.movielist
2
3 import androidx.compose.runtime.State
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.lifecycle.ViewModel
6 import androidx.lifecycle.viewModelScope
7 import com.example.movielist.data.Result
8 import com.example.movielist.data.repository.MovieRepository
9 import
10 com.example.movielist.data.repository.impl.FakeMovieRepository
11
12 import com.example.movielist.domain.model.Movie
13 import kotlinx.coroutines.flow.MutableSharedFlow
14 import kotlinx.coroutines.flow.asSharedFlow
15 import kotlinx.coroutines.launch
16 import timber.log.Timber
17
18
19 class MovieListViewModel(
20     val movieRepository : MovieRepository =
21     FakeMovieRepository()
22 ) : ViewModel() {
23
24     private val _eventFlow = MutableSharedFlow<UiEvent>()
25     val eventFlow = _eventFlow.asSharedFlow()
26
27     private val _movieListState =
28     mutableStateOf(MovieListState())
29     val movieListState: State<MovieListState> =
30     _movieListState
31
32     init {
33         getAllMovies()
34     }
35
36     fun getAllMovies() {
37         viewModelScope.launch {
38             _movieListState.value = MovieListState(isLoading
39 = true)
40
41             movieRepository.getAllMovies().collect { result
42 ->
43                 when (result) {
44                     is Result.Success<List<Movie>> -> {
45                         _movieListState.value =
46 MovieListState(movies = result.data, isLoading = false)
47
```

48	
49	Timber.tag("MovieListViewModel").i("Movies:
50	\${_movieListState.value}")
51	
52	}
53	is Result.Error -> {
54	_movieListState.value =
55	MovieListState(isLoading = false)
56	_eventFlow.emit(
57	UiEvent.ShowSnackbar(
58	message =
59	result.exception.message ?: "Unknown Error"
60	)
61	)
62	}
63	}
64	}
65	
66	
67	
68	fun onOpenUrlClicked(url: String) {
69	viewModelScope.launch {
70	if (url.isNotBlank()) {
71	_eventFlow.emit(UiEvent.OpenUrl(url))
72	Timber.tag("MovieListViewModel").i("intent
73	into URL: \$url")
74	} else {
75	_eventFlow.emit(UiEvent.ShowSnackbar("URL is
76	not valid"))
77	}
78	}
79	}
80	}
81	
82	data class MovieListState(
83	val movies: List<Movie> = emptyList(),
84	val isLoading: Boolean = true
85	)
86	
87	sealed class UiEvent {
88	data class ShowSnackbar(val message: String) : UiEvent()
89	data class OpenUrl(val url: String) : UiEvent()
90	}

Tabel 1. Source Code Jawaban Soal 1

MovieDetailViewModel.kt

1	package com.example.movielist.ui.moviedetail
2	
3	import androidx.compose.runtime.State
4	import androidx.compose.runtime.mutableStateOf


```

5      import androidx.lifecycle.ViewModel
6      import androidx.lifecycle.viewModelScope
7      import com.example.movielist.data.Result
8      import com.example.movielist.data.repository.MovieRepository
9      import
10     com.example.movielist.data.repository.impl.FakeMovieRepository
11
12     import com.example.movielist.domain.model.Movie
13     import com.example.movielist.ui.movielist.UiEvent
14     import kotlinx.coroutines.flow.MutableSharedFlow
15     import kotlinx.coroutines.flow.asSharedFlow
16     import kotlinx.coroutines.launch
17     import timber.log.Timber
18
19
20     class MovieDetailViewModel(
21         private val movieRepository: MovieRepository =
22         FakeMovieRepository(),
23     ) : ViewModel() {
24         private val _movieDetailState =
25         mutableStateOf(MovieDetailState())
26         val movieDetailState: State<MovieDetailState> =
27         _movieDetailState
28
29         private val _eventFlow =
30         MutableSharedFlow<UiEvent.ShowSnackbar>()
31         val eventFlow = _eventFlow.asSharedFlow()
32
33         fun getMovieDetail(id: Int?) {
34             if (id == null) {
35                 viewModelScope.launch {
36                     _eventFlow.emit(UiEvent.ShowSnackbar("Movie
37 ID tidak valid."))
38                 }
39                 return
40             }
41             viewModelScope.launch {
42                 _movieDetailState.value =
43                 MovieDetailState(isLoading = true)
44
45                 movieRepository.getMovie(id).collect { result ->
46                     when (result) {
47                         is Result.Success -> {
48                             _movieDetailState.value =
49                             MovieDetailState(
50                                 movie = result.data,
51                                 isLoading = false
52                             )
53
54                             Timber.tag("MovieDetailViewModel").i("Get into Movie Detail:
55 ${_movieDetailState.value}")
56

```

```

57
58         is Result.Error -> {
59             _movieDetailState.value =
60 MovieDetailState(isLoading = false)
61             _eventFlow.emit(
62                 UiEvent.ShowSnackbar(
63                     message =
64 result.exception.message ?: "Movie not found"
65                 )
66             )
67         }
68     }
69 }
70 }
71 }
72 }
73
74 data class MovieDetailState(
75     val movie: Movie? = null,
76     val isLoading: Boolean = true
77 )

```

Tabel 2. Source Code Jawaban Soal 1

XML

MainActivity.kt

```

1 package com.example.movieslistxml
2
3 import androidx.appcompat.app.AppCompatActivity
4 import android.os.Bundle
5 import androidx.navigation.NavController
6 import androidx.navigation.findNavController
7 import androidx.navigation.fragment.NavHostFragment
8 import
9 androidx.navigation.ui.setupActionBarWithNavController
10 import
11 com.example.movieslistxml.databinding.ActivityMainBinding
12
13 class MainActivity : AppCompatActivity() {
14
15     private lateinit var binding: ActivityMainBinding
16     private lateinit var navController: NavController
17
18     override fun onCreate(savedInstanceState: Bundle?) {
19         super.onCreate(savedInstanceState)
20
21         binding =
22 ActivityMainBinding.inflate(layoutInflater)
23         setContentView(binding.root)
24
25         val navHostFragment = supportFragmentManager

```

26	<code>.findFragmentById(R.id.nav_host_fragment) as</code>
27	<code>NavHostFragment // Use the ID from your XML</code>
28	<code>navController = navHostFragment.navController</code>
29	
30	<code>setupActionBarWithNavController(navController)</code>
31	<code>}</code>
32	
33	<code>override fun onSupportNavigateUp(): Boolean {</code>
34	<code>val navController =</code>
35	<code>findNavController(R.id.nav_host_fragment)</code>
36	<code>return navController.navigateUp() </code>
	<code>super.onSupportNavigateUp()</code>
	<code>}</code>
	<code>}</code>

Tabel 3. Source Code Jawaban Soal 1

MovieListAdapter.kt

1	<code>package com.example.movieslistxml</code>
2	
3	<code>import android.view.LayoutInflater</code>
4	<code>import android.view.ViewGroup</code>
5	<code>import androidx.core.content.ContextCompat.getString</code>
6	<code>import androidx.recyclerview.widget.DiffUtil</code>
7	<code>import androidx.recyclerview.widget.ListAdapter</code>
8	<code>import androidx.recyclerview.widget.RecyclerView</code>
9	<code>import com.example.movieslistxml.databinding.ItemMovieBinding</code>
10	<code>import com.example.movieslistxml.domain.MovieModel</code>
11	
12	<code>class MovieListAdapter(</code>
13	<code>private val onDetailClick: (MovieModel) -> Unit,</code>
14	<code>private val onBrowserClick: (MovieModel) -> Unit</code>
15	<code>) : ListAdapter<MovieModel,</code>
16	<code>MovieListAdapter.MovieViewHolder>(MovieDiffCallback()) {</code>
17	
18	<code>override fun onCreateViewHolder(parent: ViewGroup,</code>
19	<code>viewType: Int): MovieViewHolder {</code>
20	<code>val binding =</code>
21	<code>ItemMovieBinding.inflate(LayoutInflater.from(parent.context)</code>
22	<code>, parent, false)</code>
23	<code>return MovieViewHolder(binding)</code>
24	<code>}</code>
25	
26	<code>override fun onBindViewHolder(holder: MovieViewHolder,</code>
27	<code>position: Int) {</code>
28	<code>val movie = getItem(position)</code>
29	<code>holder.bind(movie)</code>
30	<code>}</code>
31	
32	<code>inner class MovieViewHolder(private val binding:</code>
33	<code>ItemMovieBinding) :</code>
34	<code>RecyclerView.ViewHolder(binding.root) {</code>

35	fun bind(movie: MovieModel) {
36	val context = binding.root.context
37	val titleText = getString(context,
38	movie.titleText)
39	val yearText = getString(context,
40	movie.yearText)
41	val descriptionText = getString(context,
42	movie.descriptionText)
43	
44	
45	binding.ivMoviePoster.setImageResource(movie.image)
46	binding.tvMovieTitle.text = titleText
47	binding.tvMovieYear.text = yearText
48	binding.tvMovieDescription.text =
49	descriptionText
50	binding.btnItemDetail.setOnClickListener {
51	onDetailClick(movie)
52	}
53	
54	binding.btnItemBrowser.setOnClickListener {
55	onBrowserClick(movie)
56	}
57	}
58	}
59	}
60	
61	class MovieDiffCallback :
62	DiffUtil.ItemCallback<MovieModel>() {
63	override fun areItemsTheSame(oldItem: MovieModel,
64	newItem: MovieModel): Boolean {
65	return oldItem.id == newItem.id
66	}
67	
68	override fun areContentsTheSame(oldItem: MovieModel,
69	newItem: MovieModel): Boolean {
70	return oldItem == newItem
	}
	}

Tabel 4. Source Code Jawaban Soal 1

MovieListViewModel.kt

1	package com.example.movieslistxml.ui.movieslist
2	
3	import androidx.lifecycle.LiveData
4	import androidx.lifecycle.MutableLiveData
5	import androidx.lifecycle.ViewModel
6	import androidx.lifecycle.ViewModelScope
7	import com.example.movieslistxml.data.ResultHandler
8	import
9	com.example.movieslistxml.data.repository.MovieRepository
10	import com.example.movieslistxml.domain.MovieModel

```

11 import kotlinx.coroutines.launch
12
13 class MovieListViewModel(
14     private val movieRepository: MovieRepository
15 ) : ViewModel() {
16
17     private val _movies =
18     MutableLiveData<List<MovieModel>>()
19     val movies: LiveData<List<MovieModel>> = _movies
20
21     private val _isLoading = MutableLiveData<Boolean>()
22     val isLoading: LiveData<Boolean> = _isLoading
23
24     private val _error = MutableLiveData<String?>()
25
26     init {
27         getAllMovies()
28     }
29
30     private fun getAllMovies() {
31         viewModelScope.launch {
32             _isLoading.value = true
33             movieRepository.getAllMovies().collect {
34 movieResult ->
35                 when (movieResult) {
36                     is ResultHandler.Success -> {
37                         _movies.value = movieResult.data
38                     }
39                     is ResultHandler.Error -> {
40                         _error.value =
41 movieResult.exception.message
42                     }
43                 }
44             _isLoading.value = false
45         }
46     }
47 }

```

Tabel 5. Source Code Jawaban Soal 1

MovieDetailViewModel.kt

```

1 package com.example.movieslistxml.ui.moviesdetail
2
3 import
4 com.example.movieslistxml.data.repository.MovieRepository
5
6 import androidx.lifecycle.LiveData
7 import androidx.lifecycle.MutableLiveData
8 import androidx.lifecycle.ViewModel
9 import androidx.lifecycle.viewModelScope
10 import com.example.movieslistxml.data.ResultHandler

```

```

11 import com.example.movieslistxml.domain.MovieModel
12 import kotlinx.coroutines.launch
13
14 class MovieDetailViewModel(
15     private val movieRepository: MovieRepository
16 ) : ViewModel() {
17
18     private val _movie = MutableLiveData<MovieModel?>()
19     val movie: LiveData<MovieModel?> = _movie
20
21     private val _isLoading = MutableLiveData<Boolean>()
22     val isLoading: LiveData<Boolean> = _isLoading
23
24     private val _error = MutableLiveData<String?>()
25
26     fun getMovieDetail(movieId: Int) {
27         _isLoading.value = true
28         viewModelScope.launch {
29             _isLoading.value = true
30             movieRepository.getMovie(movieId).collect {
31 movieResult ->
32                 when (movieResult) {
33                     is ResultHandler.Success -> {
34                         _movie.value = movieResult.data
35                     }
36                     is ResultHandler.Error -> {
37                         _movie.value = null
38                         _error.value =
39 movieResult.exception.message
40                     }
41                 }
42                 _isLoading.value = false
43             }
44         }
45     }
46 }

```

Tabel 6. Source Code Jawaban Soal 1

MyMovieApplication.kt

```

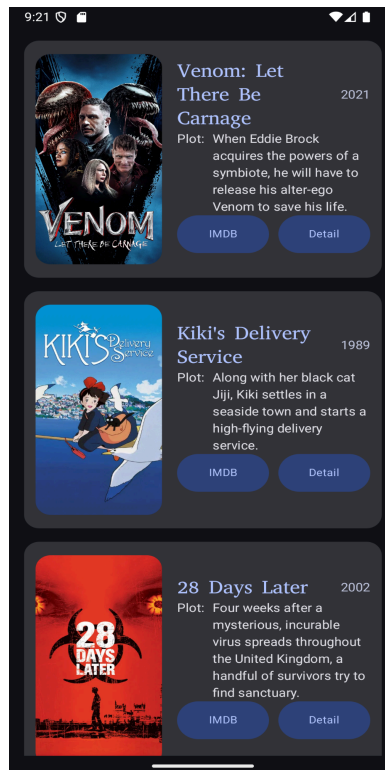
1 package com.example.movieslistxml
2
3 import android.app.Application
4 import timber.log.Timber
5
6 class MyMovieApplication : Application() {
7     override fun onCreate() {
8         super.onCreate()
9
10         // Inisialisasi Timber.
11         // Ini akan menanam DebugTree, yang secara otomatis
12         akan melakukan logging

```

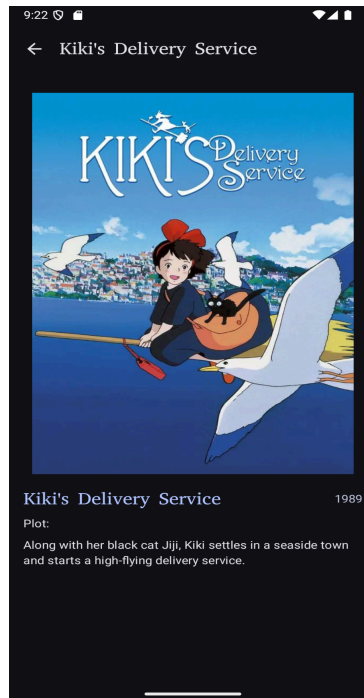
13	// dengan tag yang sesuai dan hanya aktif pada build
14	'debug'.
15	// Pada build 'release', tidak ada log yang akan
16	dicetak.
17	Timber.plant(Timber.DebugTree())
18	Timber.i("Timber is initialized.")
19	}
20	
21	}

Tabel 7. Source Code Jawaban Soal 1

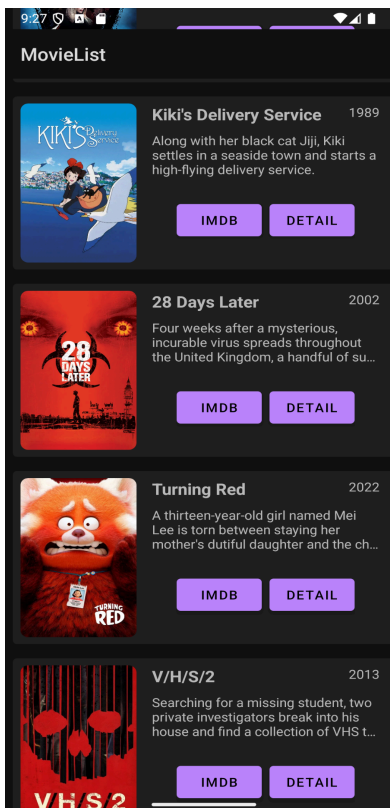
B. Output Program



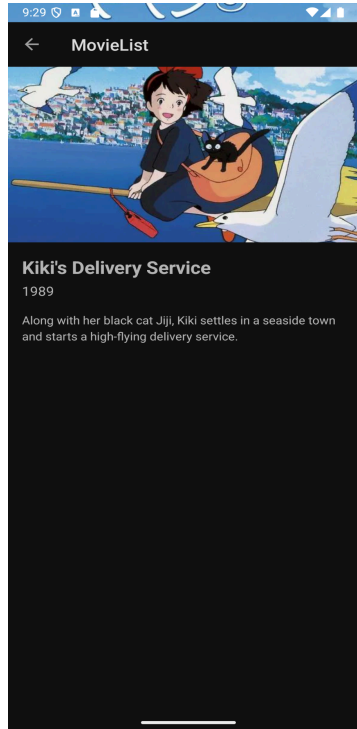
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 4. Screenshot Hasil Jawaban Soal 1 XML



Gambar 5. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

Jawaban Soal 1

compose

Pada File **MovieListViewModel** Line 36 - 66, itu adalah implementasi dan penggunaan dari **StateFlow** dimana flow yang digunakan adalah cold flow.

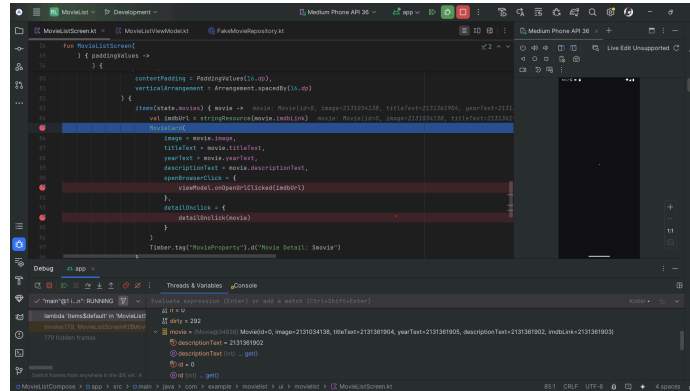
Pada File **MovieDetailViewModel** Line 33 - 70, itu adalah implementasi dan penggunaan dari **StateFlow** dimana flow yang digunakan adalah cold flow.

Pada File **MovieListViewModel** Line 49 dan 72, itu adalah implementasi dari logging menggunakan timber untuk log moviesstate value dan url pada intent.

Pada File **MovieDetailViewModel** Line 54, itu adalah implementasi dari logging menggunakan timber untuk log data dari moviedetailstate sesuai dengan yang dipilih.

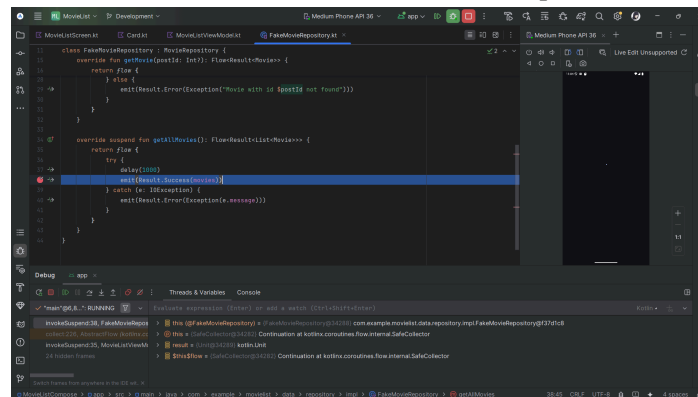
Debugger:

Breakpoint bisanya diletakan pada iterasi item ui, perubahan state ui, feching data di viewmodel, emit flow di repository.



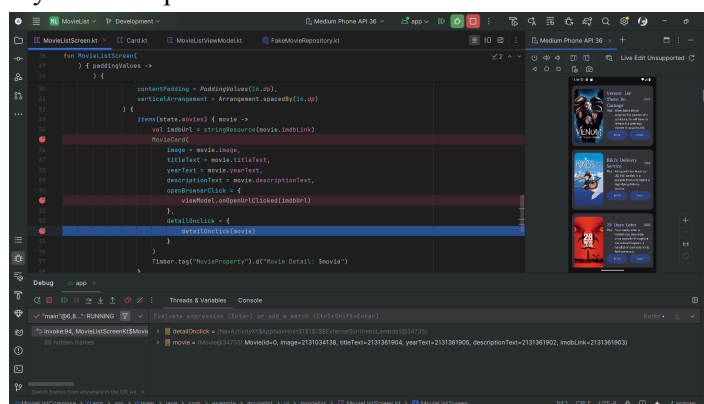
Gambar 6. Screenshot Break point dan debugger

Ini adalah posisi awal dari iterasi item ui, jika kita step out maka akan bisa dilihat bagian kode mana saja yang berjalan diatas iterasi pada saat ini sebelum iterasi selesai. Bisa dilihat fungsi dari repository perlu selesai terlebih dahulu sebelum iterasi item ui dapat selesai.



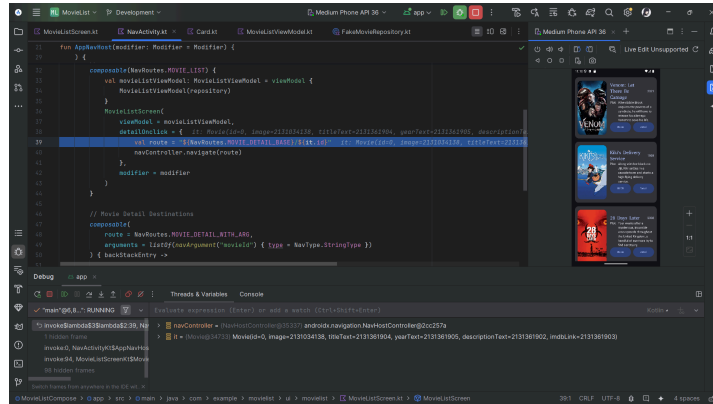
Gambar 7. Screenshot debugger step out

Ini adalah posisi dimana button detail di click kita akan step in untuk melihat kode apa yang perlu dijalankan untuk menyelesaikan proses ini.



Gambar 8. Screenshot debugger onclick

Seperti yang bisa dilihat untuk menjalankan onclick, navigasi perlu selesai terlebih dahulu, kemudian baru bisa berpindah screen.



Gambar 9. Screenshot debugger onclick stepinto

Jawaban Soal 2

Application class adalah sebuah kelas dasar (base class) dalam sebuah aplikasi Android yang digunakan untuk menjaga state atau kondisi global aplikasi. Fungsi utama dari class ini adalah one time inialisasi seperti library dan plugin. Selain itu class ini juga digunakan untuk melakukan konfigurasi tingkat aplikasi

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/biahli/Pemrograman-Mobile.git>